

Disbursement System Design Review

Created with Oboe at oboe.com

1. Problem Statement & Objectives

Currently, staff at **ChallengeAid Africa** are entrusted with cash to run operations across **51 Schools of Hope (SOH)** and **8 Virtual Learning Centers** in Kenya and Tanzania.

Managing daily expenses such as coach fees, foodstuffs, supplies, and cleaning services via physical currency introduces major institutional vulnerabilities:

- **Physical Security Risks:** Staff carrying cash in informal settlements are targets for theft and robbery.
- **Loss or Misuse Risks:** Cash transactions lack real-time visibility, delaying audit procedures.
- **Reconciliation Lag:** The finance team experiences significant delays, often discovering accounting discrepancies weeks later during physical document retirement cycles.

Core System Objective

To transition ChallengeAid Africa from a high-risk, staff-carried cash framework to a secure, tracked, digital workflow: **Digital Request** → **Multi-tier Approval** → **Bank/Mobile Disbursal** → **Automated Cryptographic Reconciliation**.

2. System Architecture & High-Level Workflow

```
[Programme Officer]
  | (Submits Request & Uploads Receipt)
  ▼
[PostgreSQL: PaymentRequests (Status: Submitted)]
  |
  ▼
[Finance / Accounts Officer Approval]
```


The Core Execution Stages

- **Request Creation:** Staff-in-charge inputs SOH centre details, budget line, payment type, recipient details, amount, justification, and initially available attachments.
 - **Finance Review:** Verifies budget allocation and document completeness, transitioning the request status.
 - **Director Approval:** Primary leadership sign-off.
 - **Dynamic Threshold Routing:** The rules engine evaluates the request. If the transaction is under KES 10,000, trustee approvals are bypassed. Otherwise, it triggers the multi-trustee routing.
 - **Trustee Sign-off Chain:** Requires all 4 trustees to sign off asynchronously if the threshold is met.
 - **Disbursement:** Executed manually (Phase 1) or via integrated bank gateways (Phase 2).
 - **Reconciliation:** Staff upload supporting materials. The system validates documentation rules by payment type and updates status to **Reconciled**.
-

3. Database Schema & State Management

To ensure complete referential integrity and support parallel, multi-user operations, the PostgreSQL schema is fully normalized. It utilizes specific column constraints, check rules, and row-level locks.

```
-- Core Enumerated Types

CREATE TYPE user_role AS ENUM ('Programme Officer', 'Finance Officer', 'Director',
'Trustee', 'System Admin');

CREATE TYPE payment_type AS ENUM ('Coach Fees', 'Foodstuffs', 'Supplies', 'Cleaning
Services', 'Other');

CREATE TYPE request_status AS ENUM ('Draft', 'Submitted', 'Finance Approved',
'Director Approved', 'Trustee Pending', 'Approved', 'Disbursed', 'Reconciled',
'Rejected', 'More Information Requested');
```

```
CREATE TYPE doc_type AS ENUM ('Signed Payment Form', 'Coach Signature', 'Vendor Invoice', 'Delivery Note', 'Service Confirmation', 'Other Evidence');
```

```
-- Centres Table
```

```
CREATE TABLE centres (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    location VARCHAR(50) NOT NULL CHECK (location IN ('Kenya', 'Tanzania'))  
);
```

```
-- Budget Lines Table
```

```
CREATE TABLE budget_lines (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    allocated_amount DECIMAL(15, 2) NOT NULL CHECK (allocated_amount >= 0),  
    spent_to_date DECIMAL(15, 2) NOT NULL DEFAULT 0.00 CHECK (spent_to_date <=  
allocated_amount)  
);
```

```
-- Users Table
```

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    role user_role NOT NULL,  
    centre_id INT REFERENCES centres(id),  
    contact_info VARCHAR(100) NOT NULL,  
    is_active BOOLEAN DEFAULT TRUE  
);
```

```
-- Payment Requests Table with Optimistic Locking Column
```

```
CREATE TABLE payment_requests (  
    id SERIAL PRIMARY KEY,  
    requester_id INT NOT NULL REFERENCES users(id),  
    centre_id INT NOT NULL REFERENCES centres(id),  
    budget_line_id INT NOT NULL REFERENCES budget_lines(id),  
    payment_category payment_type NOT NULL,  
    recipient_name VARCHAR(150) NOT NULL,  
    recipient_phone VARCHAR(50) NOT NULL,
```

```

    recipient_bank_acc VARCHAR(50),
    amount DECIMAL(15, 2) NOT NULL CHECK (amount > 0),
    justification TEXT NOT NULL,
    status request_status NOT NULL DEFAULT 'Draft',
    version INT NOT NULL DEFAULT 1,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Approvals Ledger Table (Capturing Sequential Sign-offs)
CREATE TABLE approvals (
    id SERIAL PRIMARY KEY,
    request_id INT NOT NULL REFERENCES payment_requests(id) ON DELETE CASCADE,
    approver_id INT NOT NULL REFERENCES users(id),
    role_at_approval user_role NOT NULL,
    decision VARCHAR(20) NOT NULL CHECK (decision IN ('Approved', 'Rejected', 'Info
Requested')),
    comments TEXT,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- System Rules Table for Policy-Driven Routing
CREATE TABLE system_rules (
    key VARCHAR(50) PRIMARY KEY,
    value_num DECIMAL(15, 2),
    description TEXT
);

INSERT INTO system_rules (key, value_num, description)
VALUES ('TRUSTEE_BYPASS_THRESHOLD', 10000.00, 'Requests under this KES value bypass
Trustee steps.');
```

Preventing Race Conditions via Optimistic and Pessimistic Locking

1. Pessimistic Locking (**SELECT FOR UPDATE**)

When a trustee or executive clicks "Approve", the system locks the target

payment_requests record immediately to prevent concurrent requests from writing

conflicting states at the exact same millisecond.

```
-- Executed inside a standard PostgreSQL Transaction block
BEGIN;

-- Lock the selected payment request row exclusively until transaction completes
SELECT id, status, amount, version
FROM payment_requests
WHERE id = 42
FOR UPDATE;

-- Validate current state and proceed with business state logic...
```

2. Optimistic Locking (Application Level Safeguard)

To prevent two trustees from concurrently updating the database after viewing the same stale web state, the backend uses a version checking pattern:

```
-- Attempt to write the approval change only if the version hasn't changed
UPDATE payment_requests
SET status = 'Approved',
    version = version + 1,
    updated_at = CURRENT_TIMESTAMP
WHERE id = 42 AND version = 3;

-- If 0 rows are affected, the application rolls back and alerts the user of a state
conflict.
```

4. Business Rules & Dynamic Threshold Routing Engine

To resolve the operational bottleneck where trustees must review trivial disbursements, the database uses a rule-driven state transition function. This function bypasses trustee approval steps for minor operational expenses under KES 10,000.

```

CREATE OR REPLACE FUNCTION process_approval_transition(p_request_id INT,
p_approver_id INT, p_decision VARCHAR(20), p_comments TEXT)
RETURNS VOID AS $$
DECLARE
    v_current_status request_status;
    v_amount DECIMAL(15, 2);
    v_threshold DECIMAL(15, 2);
    v_approver_role user_role;
    v_trustee_count INT;
BEGIN
    -- Lock target rows
    SELECT status, amount INTO v_current_status, v_amount
    FROM payment_requests
    WHERE id = p_request_id
    FOR UPDATE;

    SELECT role INTO v_approver_role
    FROM users
    WHERE id = p_approver_id;

    SELECT value_num INTO v_threshold
    FROM system_rules
    WHERE key = 'TRUSTEE_BYPASS_THRESHOLD';

    -- Insert the new approval signature record
    INSERT INTO approvals (request_id, approver_id, role_at_approval, decision,
comments)
    VALUES (p_request_id, p_approver_id, v_approver_role, p_decision, p_comments);

    -- Process Rejections Immediately
    IF p_decision = 'Rejected' THEN
        UPDATE payment_requests SET status = 'Rejected' WHERE id = p_request_id;
        RETURN;
    ELSIF p_decision = 'Info Requested' THEN
        UPDATE payment_requests SET status = 'More Information Requested' WHERE id =
p_request_id;
        RETURN;

```

```

END IF;

-- Evaluate State Machine Logic
CASE v_current_status
    WHEN 'Submitted' THEN
        IF v_approver_role = 'Finance Officer' THEN
            UPDATE payment_requests SET status = 'Finance Approved' WHERE id =
p_request_id;
        END IF;

        WHEN 'Finance Approved' THEN
            IF v_approver_role = 'Director' THEN
                -- Evaluate Dynamic Threshold Routing
                IF v_amount < v_threshold THEN
                    -- Bypass Trustee Approval States entirely
                    UPDATE payment_requests SET status = 'Approved' WHERE id =
p_request_id;
                ELSE
                    UPDATE payment_requests SET status = 'Trustee Pending' WHERE id =
p_request_id;
                END IF;
            END IF;

            WHEN 'Trustee Pending' THEN
                IF v_approver_role = 'Trustee' THEN
                    -- Verify total unique trustee signatures recorded
                    SELECT COUNT(DISTINCT approver_id) INTO v_trustee_count
                    FROM approvals
                    WHERE request_id = p_request_id AND role_at_approval = 'Trustee' AND
decision = 'Approved';

                    IF v_trustee_count >= 4 THEN
                        UPDATE payment_requests SET status = 'Approved' WHERE id =
p_request_id;
                    END IF;
                END IF;
            ELSE
                RAISE EXCEPTION 'Invalid State Transition Attempted';

```



```
END CASE;  
  
END;  
  
$$ LANGUAGE plpgsql;
```

5. Security, Cryptographic Tamper-Evidence & Verification

To guarantee ledger integrity against malicious or unauthorized database-level writes, the system maintains a chronologically chained audit log. Every record links to the previous entry using a cryptographic hash.

```
CREATE TABLE audit_trail (  
  id SERIAL PRIMARY KEY,  
  timestamp TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,  
  action VARCHAR(100) NOT NULL,  
  payload JSONB NOT NULL,  
  previous_hash VARCHAR(64) NOT NULL,  
  current_hash VARCHAR(64) NOT NULL UNIQUE  
);
```

The Chaining Principle

For each entry n , the system calculates:

$$\text{Hash}_n = \text{SHA-256}(\text{Payload}_n \parallel \text{Timestamp}_n \parallel \text{Hash}_{n-1})$$

If any transaction value or historical record is altered, its individual hash changes. This breaks the sequential chain, instantly alerting system auditors to database manipulation.

Manual Execution Safety: Regex Pattern Matching

During Phase 1 (Manual Banking Execution), the backend validates manually entered bank transaction reference numbers against strict regex rules before updating statuses:

```
// Regex to validate standard Cooperative Bank of Kenya (M-Co-op Cash / M-Pesa)  
reference IDs  
  
const bankTxRegex = /^[A-Z0-9]{10,12}$/;
```

```
function validateTransactionReference(referenceInput) {
  if (!bankTxRegex.test(referenceInput)) {
    throw new Error("Invalid transaction reference string format.");
  }
  return true;
}
```

6. Payment Integration & Webhook Security (Phase 2)

In Phase 2, manual validation is replaced by direct API integration with Cooperative Bank and mobile money APIs. The server exposes a secure endpoint to receive transaction notifications and utilizes HMAC-SHA256 signature verification to prevent spoofing.

HMAC-SHA256 Verification Pattern

```
const crypto = require('crypto');

function verifyBankWebhook(req, res, next) {
  const sharedSecret = process.env.BANK_WEBHOOK_SECRET;
  const incomingSignature = req.headers['x-bank-signature'];

  if (!incomingSignature) {
    return res.status(401).json({ error: 'Signature header missing' });
  }

  // Compute the expected HMAC using the raw request body and shared secret
  const calculatedSignature = crypto
    .createHmac('sha256', sharedSecret)
    .update(JSON.stringify(req.body))
    .digest('hex');

  if (crypto.timingSafeEqual(Buffer.from(incomingSignature, 'hex'),
    Buffer.from(calculatedSignature, 'hex'))) {
    next();
  }
}
```

```
    } else {  
        return res.status(403).json({ error: 'Cryptographic signature mismatch' });  
    }  
}
```

7. Asynchronous Notification Pipeline

To ensure the platform remains responsive in low-bandwidth or intermittent connectivity environments, expensive external operations — such as sending SMS notifications — are decoupled from the HTTP request-response cycle using an asynchronous queue framework.

```
[Client Request]  
    | (Triggers approval)  
    ▼  
[HTTP Controller] —> Updates DB immediately & returns 200 OK  
    |  
    ▼ (Enqueues Notification Job)  
[Redis Queue (BullMQ)]  
    |  
    ▼  
[Background Worker Thread]  
    |  
    └─ (Success) —> Sends SMS via Cellular Gateway API  
    |  
    └─ (Failure / Timeout) —> Exponential Backoff Retry Loop
```

- **Performance Isolation:** If cellular gateway networks experience high latency or downtime in East Africa, the application's core user interface remains unaffected.
 - **Queue Durability:** Outgoing SMS messages are held in Redis, with custom retry strategies ensuring delivery when connectivity stabilizes.
-


```

| • Budget vs Actual |
| • Establishes hygiene | → | • Dynamic thresholds | → | • SMS worker queue |
→ | reporting metrics |
| • Zero development | | • Manual bank uploads | | • Mobile optimization |
| • Board export tools |
|_____| |_____| |_____|
|_____|

```

Phase 1 Foundation Priorities

Before implementing external payment gateways or third-party automated networks, the development team must finalize the following foundations:

- **PostgreSQL Relational Schema:** Enforcing standard integrity, primary keys, and index configurations.
- **Role-Based Access Control (RBAC):** Restricting visibility to program data based on user roles.
- **Cryptographically Signed Audit Logs:** Establishing an unalterable history of financial activity.